Тестовое задание — C/ .NET Backend Developer (Middle)

Привет! Это тестовое для позиции Middle.
Рассчитываю, что ты справишься за 1–1.5 часа. Если что-то не успеешь — напиши, что хотел доделать.
Код должен быть рабочим, но не обязательно идеальным. Грязные хаки, TODO и FIXME — можно, если пояснишь.

Важно: мы потом обсудим код по видео, так что лучше пусть будет неидеально, но твоё.

Технологии (без сюрпризов)

- .NET 9
- ASP.NET Core Minimal API
- PostgreSQL + EF Core (миграции приветствуются, но не обязательно)
- RabbitMQ (только publish, consumer не нужен)

Можно использовать Docker, но не обязательно.

Что нужно сделать:

Сущность TaskItem (примерно такая)

```csharp
public class TaskItem
{
    public Guid Id { get; set; }
    public string Title { get; set; } // не null, не пустой, максимум 200
    public bool IsCompleted { get; set; }
    public DateTimeOffset CreatedAt { get; set; }
    public DateTimeOffset? CompletedAt { get; set; } // когда завершили
    public Priority Priority { get; set; } // Low, Medium, High
}
```

Важно: поле `CompletedAt` должно заполняться только в момент завершения.

API

Метод	Эндпоинт	Что делает
POST	`/tasks`	создаёт задачу (по умолчанию IsCompleted=false, Priority=Medium)
GET	`/tasks`	возвращает **все** задачи (без пагинации — упростим)

| PUT | `/tasks/{id}/complete` | завершает задачу, ставит CompletedAt, отправляет RabbitMQ |
| DELETE | `/tasks/{id}` | удаляет задачу (жестко, без восстановления) |

> Почему без пагинации? Потому что в реальном проекте её добавили бы через 2 недели.
> Если сделаешь — плюс, но не обязательно.

Валидация
- `Title` — не пустой, не больше 200 символов
- При создании задачи, если Title некорректен — вернуть 400
- Остальную валидацию (enum, guid) сделай через модель, но без фанатизма

RabbitMQ
При завершении задачи нужно отправить сообщение в **обменник `task.events`** с routing key `task.completed`.

Сообщение:

```json
{
  "taskId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "title": "Buy milk",
  "completedAt": "2026-05-07T12:00:00Z",
  "priority": "High"
}
```

Что я хочу увидеть в коде:
- Публикация без блокировки основного потока (не надо ждать confirms, если не успеваешь)
- Если RabbitMQ недоступен — задачу всё равно завершаем, а сообщение пишем в лог (fail silently, но с логом)

> Да, это не идеально по DDD, но в 80% проектов так и делают.

Конкурентность
Если два запроса одновременно вызывают `/tasks/{id}/complete`:

- Первый завершает задачу
- Второй должен получить 409 Conflict

Используй для этого RowVersion в EF Core (человеческий способ без локов).

Если не знаешь как — можно обойтись проверкой `IsCompleted` перед сохранением (но будет race condition).
Я замечу, но сильно ругать не буду.

Graceful shutdown
При остановке приложения (Ctrl+C, SIGTERM):
- RabbitMQ канал и соединение должны корректно закрыться
- Публикации, которые уже начались, должны успеть отправиться (таймаут 3-5 секунд)

Остальное (БД, Kestrel) .NET закроет сам — не парься.

Тесты
Напиши один интеграционный тест:

```
1. Создать задачу через API
2. Завершить её
3. Проверить, что RabbitMQ получил сообщение (через TestConsumer или мок)
```

Если не успеваешь с RabbitMQ — заглуши его интерфейсом и проверь вызов.
Я пойму.

Чего я не буду требовать:
- Пагинация / фильтрация / сортировка (если сделаешь — плюс)
- Unit-тесты (один интеграционный — ок)
- Swagger
- Валидация через FluentValidation (можно через if)
- Аутентификация
- Логирование сверх стандартного
- Docker Compose

Что я обязательно замечу:
- Title пропускает пустые строки → минус
- CompletedAt заполняется при создании (а не при завершении) → минус
- RabbitMQ отправляет сообщение ДО сохранения в БД (если БД упадёт — сообщение уйдёт зря) → это плохо, но не смертельно
- Отсутствие проверки на 409 при конкурентных вызовах → замечу

Как я буду оценивать:
Критерий	Вес
Всё работает (API, БД, RabbitMQ)	30%
Обработка ошибок и валидация	20%
Работа с конкурентностью (rowversion)	15%
RabbitMQ + graceful shutdown	15%
Интеграционный тест	10%
Чистота кода (читаемость, структура)	10%

Структура проекта

Можно всё в одном `Program.cs`, но с `#region` или partial-классами.
Главное — чтобы я нашёл нужные места без поиска Ctrl+F.

Пример:

```
TaskApi/
├── Program.cs
├── TaskItem.cs
├── TaskDbContext.cs
├── RabbitMqService.cs
├── appsettings.json
└── Tests/
    └── CompleteTaskIntegrationTest.cs
```

Напоследок

Я знаю, что за час идеально не напишешь.
Я хочу увидеть, как ты расставляешь приоритеты и где готов срезать угол.

Если что-то не успеваешь — добавь комментарий `// HACK: не успел, доделал бы так: ...`.

Удачи!
Если будут вопросы — пиши @lisovaaadi, но лучше сам принимай решения (как в реальной работе).